DAVE3606 - Assignment

1.

These are the SQL statements that were used to create primary keys to the database tables:

ALTER TABLE lego_set ADD PRIMARY KEY (id);
ALTER TABLE lego_brick ADD CONSTRAINT PK_Lego_Brick PRIMARY KEY (brick_type_id, color_id);
ALTER TABLE lego_inventory ADD PRIMARY KEY (set_id, brick_type_id, color_id);

And these are the Foreign key constraints that were added:

ALTER TABLE lego_inventory ADD FOREIGN KEY (set_id) REFERENCES lego_set(id);

ALTER TABLE lego_inventory ADD FOREIGN KEY (brick_type_id, color_id) REFERENCES lego_brick(brick_type_id, color_id);

I chose to use (brick_type_id, color_id) since it's more natural to search for a specific brick type and then filter by color. Same for lego_inventory (set_id, brick_type_id, color_id), the most common query will be to fetch all bricks for a given set, which is why set_id is the leading column.

2.

Queries used:

CREATE INDEX ON lego_inventory(brick_type_id);
CREATE INDEX ON lego_inventory(color_id);

```
lego-db=# \timing on
Timing is on.
lego-db=# SELECT set_id FROM lego_inventory WHERE brick_type_id = '3001';
Time: 193.090 ms
lego-db=# SELECT set_id FROM lego_inventory WHERE color_id = 9;
Time: 74.309 ms
lego-db=#
```

```
lego-db=# SELECT set_id FROM lego_inventory WHERE brick_type_id = '3001';
Time: 127.023 ms
lego-db=# SELECT set_id FROM lego_inventory WHERE color_id = 9;
Time: 61.183 ms
```

As we can see from the pictures we can see the query performance improved to the the database being able to use a B-tree index to jump directly to the relevant rows, instead of performing a scan through the entire table.

3. The time complexity of the original code results in O(n^2) due to repeated string concatenations used inside a loop. This was fixed by collecting rows in a list and joining them once after the loop, reducing the complexity to O(n).

5. Set id stored as a 2-byte big-endian to indicate string length followed by the UTF-8 encoded string bytes. Set name is stored the same way. Year is stored as a 2-byte integer, the category as a length-prefixed string, and a 4-byte integer indicating the number of inventory items. Each inventory item consists of 2-bytes for the length of brick_type_id, 2-bytes for color_id and 2-bytes for count.

6.

```
127.0.0.1 - - [27/Mar/2026 13:32:00] "GET / HTTP/1.1" 200 -
Time to render all sets: 0.09045474599406589
127.0.0.1 - - [27/Mar/2026 13:32:03] "GET /sets HTTP/1.1" 200 -
127.0.0.1 - - [27/Mar/2026 13:32:07] "GET /set?id=00-1 HTTP/1.1" 200 -
Cache MISS: 0.004326s
127.0.0.1 - - [27/Mar/2026 13:32:07] "GET /api/set?id=00-1 HTTP/1.1" 200 -
127.0.0.1 - - [27/Mar/2026 13:32:12] "GET /set?id=00-1 HTTP/1.1" 200 -
Cache HIT: 0.000110s
127.0.0.1 - - [27/Mar/2026 13:32:12] "GET /api/set?id=00-1 HTTP/1.1" 200 -
```

For task 6 a server-side LRU cache was implemented using OrderedDict. When a user requests a set, the cache is checked first, if the cache hits the set is moved to the end using the method move_to_end(), telling the server its been recently used, and then the cached result returns immediately. On a cache miss, we query the databass and store the result in the cache. After 100 entries the cache evicts the least recently used entry.

The complexity of both cache hits and eviction is O(1).

The measured response times for a Cache Miss is 0.004326s compared to a cache hit of 0.000110s, making the response time 40x faster during a cache hit.

We also cache the page for up to 60 seconds.